

UNIT– I: JAVA BASICS

1. What is Java?

Answer: Java is a high-level, object-oriented, class-based programming language developed by Sun Microsystems. It is designed to be portable and platform independent through the JVM.

2. Who developed Java?

Answer: Java was developed by **James Gosling and his team at Sun Microsystems**.

3. When was Java introduced?

Answer: Java was publicly released in **1995**.

4. What was Java originally called?

Answer: Java was originally called **Oak**.

5. What is the main principle of Java?

Answer: Write Once, Run Anywhere (WORA).

6. Why is Java platform independent?

Answer: Java source code is compiled into **bytecode**, which can run on any platform having a compatible JVM.

7. What is bytecode?

Answer: Bytecode is the intermediate code generated by the Java compiler. It is stored in a .class file and executed by the JVM.

8. What is JVM?

Answer: JVM stands for **Java Virtual Machine**. It executes Java bytecode and provides platform independence.

9. What is JDK?

Answer: JDK stands for **Java Development Kit**. It contains tools required to develop, compile, debug and run Java programs.

10. What is JRE?

Answer: JRE stands for **Java Runtime Environment**. It provides the environment required to run Java applications.

11. Difference between JDK, JRE and JVM?

Answer:

- **JDK** → Develops Java applications.
- **JRE** → Runs Java applications.
- **JVM** → Executes Java bytecode.

12. What are Java buzzwords?

Answer: Important Java features include **simple, object-oriented, platform independent, portable, secure, robust, multithreaded, distributed, architecture-neutral, high-performance and dynamic**.

Data Types and Variables

13. What is a data type?

Answer: A data type specifies what kind of value a variable can store.

14. How many primitive data types are there in Java?

Answer: There are **8 primitive data types**: byte, short, int, long, float, double, char and boolean.

15. What is the default value of an int instance variable?

Answer: 0.

16. What is the default value of a boolean instance variable?

Answer: false.

17. What is the default value of a reference variable?

Answer: null.

18. What is a variable?

Answer: A variable is a named memory location used to store a value.

19. What are the types of variables in Java?

Answer: Local variables, instance variables and static/class variables.

20. What is a local variable?

Answer: A variable declared inside a method, constructor or block is called a local variable.

21. What is an instance variable?

Answer: A non-static variable declared inside a class but outside methods is called an instance variable.

22. What is a static variable?

Answer: A variable declared using static that belongs to the class rather than individual objects.

23. What is variable scope?

Answer: Scope is the region of a program where a variable can be accessed.

Arrays

24. What is an array?

Answer: An array is a fixed-size collection of elements of the same type stored under one variable name.

25. Does Java support dynamic arrays?

Answer: Java arrays have fixed length. For dynamically sized collections, we can use classes such as ArrayList.

26. What is the first index of an array?

Answer: The first index is **0**.

27. How do you find the length of an array?

Answer:

`array.length()`

28. Can an array store different data types?

Answer: Normally, an array stores elements of one declared type, although a reference-type array can hold objects of compatible subclasses.

Operators and Control Statements

29. What is an operator?

Answer: An operator is a symbol that performs an operation on one or more operands.

30. Name some types of operators in Java.

Answer: Arithmetic, relational, logical, assignment, unary, bitwise, shift and conditional operators.

31. What is the difference between == and =?

Answer:

- = is the assignment operator.
- == compares values or, for references, checks whether two references refer to the same object.

32. What is the difference between && and &?

Answer: && is the logical AND operator with short-circuit evaluation. & can perform bitwise AND and can also act as a non-short-circuit boolean AND.

33. What are control statements?

Answer: Control statements determine the order in which statements are executed.

34. What are the types of control statements?

Answer:

1. Selection
2. Iteration
3. Jump statements

35. Give examples of selection statements.

Answer: if, if-else, else-if and switch.

36. Give examples of looping statements.

Answer: for, while and do-while.

37. What is the difference between while and do-while?

Answer: while checks the condition before execution, whereas do-while executes the body at least once before checking the condition.

Type Conversion and Casting

38. What is type conversion?

Answer: Type conversion means converting a value from one data type to another.

39. What is widening conversion?

Answer: Conversion from a smaller compatible numeric type to a larger type.

```
int a = 10;  
double b = a;
```

40. What is narrowing conversion?

Answer: Conversion from a larger type to a smaller type using explicit casting.

```
double a = 10.5;  
int b = (int)a;
```

41. What is type casting?

Answer: Explicitly converting one data type into another using a cast operator.

Classes, Objects and Methods

42. What is a class?

Answer: A class is a blueprint or template for creating objects.

43. What is an object?

Answer: An object is an instance of a class.

44. How do you create an object in Java?

Answer:

```
Student s = new Student();
```

45. What is the new keyword?

Answer: new is used to create objects and allocate memory for them.

46. What is a method?

Answer: A method is a block of code that performs a specific operation.

47. Why are methods used?

Answer: Methods provide **code reuse, modularity, readability and easier maintenance.**

48. Can a method return multiple values?

Answer: A method has one declared return type, but multiple values can be returned together using an object, array, collection, etc.

Constructors

49. What is a constructor?

Answer: A constructor is a special member used to initialize an object. It has the same name as the class and has no return type.

50. When is a constructor called?

Answer: It is called automatically when an object is created.

51. Can a constructor be inherited?

Answer: No, constructors are not inherited.

52. Can constructors be overloaded?

Answer: Yes.

```
Student() { }
```

```
Student(int age) { }
```

53. Can a constructor be static?

Answer: No. Constructors belong to object creation and cannot be declared static.

54. Can a constructor be final?

Answer: No. Constructors cannot be overridden, so final is not applicable to them.

Access Control

55. What are access modifiers in Java?

Answer: private, default/package-private, protected and public.

56. Which is the most restrictive access modifier?

Answer: private.

57. Which is the least restrictive?

Answer: public.

58. What is the purpose of access modifiers?

Answer: They control the visibility and accessibility of classes, variables, methods and constructors.

this Keyword

59. What is this?

Answer: this refers to the current object.

60. Why is this commonly used in constructors?

Answer: It is commonly used to distinguish instance variables from parameters having the same name.

```
Student(int age) {  
    this.age = age;  
}
```

61. Can this be used in a static method?

Answer: No, because this refers to an object and static methods belong to the class rather than a particular object.

Garbage Collection

62. What is garbage collection?

Answer: Garbage collection is the automatic process of reclaiming memory occupied by objects that are no longer reachable.

63. Who performs garbage collection?

Answer: The **JVM's garbage collector**.

64. Can we force garbage collection?

Answer: No. We can request it using:

```
System.gc();
```

but the JVM is not required to perform it immediately.

65. When does an object become eligible for garbage collection?

Answer: When there are no reachable references to that object.

Parameter Passing

66. Is Java pass-by-value or pass-by-reference?

Answer: Java is **strictly pass-by-value**.

67. What happens when an object is passed to a method?

Answer: The **reference value is copied and passed by value**. Through that copied reference, the method can modify the object's fields.

68. Can a method change the caller's object reference?

Answer: No. Reassigning the parameter only changes the local copy of the reference.

Recursion

69. What is recursion?

Answer: Recursion is a technique in which a method calls itself.

70. What is essential in recursion?

Answer: A **base condition** is essential to stop recursive calls.

71. Give an example of recursion.

Answer: Factorial, Fibonacci series, tree traversal and binary search can be implemented recursively.

String

72. What is String in Java?

Answer: String is a class used to represent a sequence of characters.

73. Is String mutable or immutable?

Answer: String objects are **immutable**.

74. Why is String immutable?

Answer: Immutability provides benefits such as security, thread-safety characteristics, string pooling and predictable behavior.

75. How do you compare two strings?

Answer: Use equals() to compare their contents.

```
s1.equals(s2)
```

76. What is the difference between == and equals() for String?

Answer:

- == compares references.
- equals() compares string contents.

77. Name some String methods.

Answer: length(), charAt(), substring(), equals(), replace(), toUpperCase() and toLowerCase().

Collections

78. What is the Java Collections Framework?

Answer: It is a set of interfaces and classes used to store and manipulate groups of objects.

79. What is ArrayList?

Answer: ArrayList is a resizable array implementation of the List interface.

80. Does ArrayList allow duplicates?

Answer: Yes.

81. Does ArrayList maintain insertion order?

Answer: Yes.

82. What is HashMap?

Answer: HashMap stores data as **key-value pairs**.

83. Can HashMap have duplicate keys?

Answer: No. A key is unique. If the same key is inserted again, its associated value is replaced.

84. Can HashMap have duplicate values?

Answer: Yes.

85. Does HashMap guarantee insertion order?

Answer: No.

86. Difference between ArrayList and HashMap?

ArrayList	HashMap
Stores elements	Stores key-value pairs
Uses index	Uses key
Allows duplicate elements	Keys are unique
Maintains insertion order	Does not guarantee insertion order

UNIT– II: INHERITANCE

87. What is inheritance?

Answer: Inheritance is an OOP mechanism through which a subclass acquires accessible members and behavior from a superclass.

88. Which keyword is used for inheritance?

Answer: extends.

```
class Dog extends Animal
```

89. What are the types of inheritance supported through Java classes?

Answer: Single, multilevel and hierarchical inheritance.

90. Does Java support multiple inheritance using classes?

Answer: No. Java does not support multiple inheritance of classes.

91. How can Java achieve multiple inheritance of type?

Answer: Through interfaces, because a class can implement multiple interfaces.

super Keyword

92. What is super?

Answer: super refers to the immediate parent class.

93. What are the uses of super?

Answer:

1. Access parent class variable.
2. Call parent class method.
3. Call parent class constructor.

94. Can super() be used anywhere in a constructor?

Answer: No. A constructor invocation such as super() must be the first statement in the constructor.

final

95. What is the final keyword?

Answer: final prevents further modification in different contexts.

96. What happens if a variable is final?

Answer: It cannot be reassigned after initialization.

97. Can a final method be overridden?

Answer: No.

98. Can a final class be inherited?

Answer: No.

99. Give an example of a final class.

Answer: String is a well-known example of a final class in Java.

Object Class

100. What is the Object class?

Answer: Object is the root superclass of all Java classes.

101. Name some methods of Object class.

Answer: toString(), equals(), hashCode(), getClass(), wait(), notify() and notifyAll().

102. What is the purpose of toString()?

Answer: It returns a string representation of an object.

103. What is the purpose of equals()?

Answer: It is used to compare objects for logical equality, depending on the class's implementation.

104. What is the purpose of hashCode()?

Answer: It returns a hash value used by hash-based collections such as HashMap and HashSet.

POLYMORPHISM

105. What is polymorphism?

Answer: Polymorphism means **one interface or method name can represent different forms of behavior.**

106. What are the two major types of polymorphism?

Answer:

1. Compile-time polymorphism – method overloading
2. Runtime polymorphism – method overriding

107. What is method overloading?

Answer: Defining multiple methods with the same name but different parameter lists in the same class.

108. Can we overload a method by changing only its return type?

Answer: No.

```
int add(int a, int b)
double add(int a, int b) // Invalid overload
```

109. What is method overriding?

Answer: When a subclass provides its own implementation of an inherited overridable instance method.

110. What annotation is commonly used for overriding?

Answer:

```
@Override
```

111. What is dynamic binding?

Answer: Dynamic binding is the process where the overridden method to execute is selected at runtime based on the actual object.

112. Give an example of runtime polymorphism.

```
Animal a = new Dog();
a.sound();
```

Answer: If Dog overrides sound(), the Dog version executes at runtime.

ABSTRACT CLASS

113. What is an abstract class?

Answer: A class declared with abstract that may contain abstract and concrete methods.

114. Can we create an object of an abstract class?

Answer: No, an abstract class cannot be directly instantiated.

115. Can an abstract class have a constructor?

Answer: Yes.

116. Can an abstract class contain concrete methods?

Answer: Yes.

117. What is an abstract method?

Answer: A method declared without an implementation.

```
abstract void display();
```

PACKAGES

118. What is a package?

Answer: A package is a namespace used to organize related classes and interfaces.

119. Which keyword is used to create a package?

Answer:

```
package
```

120. Which keyword is used to access a class from another package?

Answer:

import

121. Why are packages used?

Answer: For organization, namespace management, access control and avoiding naming conflicts.

122. What is the default package?

Answer: A class without an explicit package declaration belongs to the unnamed/default package.

INTERFACES

123. What is an interface?

Answer: An interface defines a contract that implementing classes agree to follow.

124. Which keyword is used to implement an interface?

Answer:

implements

125. Which keyword is used when one interface inherits another?

Answer:

extends

126. Can a class implement multiple interfaces?

Answer: Yes.

```
class Test implements A, B {  
}
```

127. Can an interface extend another interface?

Answer: Yes.

128. Can an interface extend multiple interfaces?

Answer: Yes.

```
interface C extends A, B {  
}
```

129. What are interface variables by default?

Answer: They are implicitly **public, static and final**.

130. Can we modify an interface variable?

Answer: No, because it is implicitly final.

FUNCTIONAL INTERFACE AND LAMBDA

131. What is a functional interface?

Answer: An interface with exactly **one abstract method**.

132. Which annotation is used for functional interfaces?

Answer:

```
@FunctionalInterface
```

133. What is a lambda expression?

Answer: A lambda expression is a concise way to provide an implementation of a functional interface.

134. Give the syntax of a lambda expression.

```
(parameters) -> expression
```

135. What is the main advantage of lambda expressions?

Answer: They reduce boilerplate code and make functional-style programming more concise.

136. In which Java version were lambda expressions introduced?

Answer: Java 8.

UNIT– III: EXCEPTION HANDLING

137. What is an exception?

Answer: An exception is an abnormal condition during program execution that can disrupt the normal flow of a program .

138. What is exception handling?

Answer: Exception handling is a mechanism for detecting and handling runtime problems so that the program can respond appropriately.

139. What is the root class of Java exceptions and errors?

Answer: Throwable.

140. What are the two major branches under Throwable?

Answer:

```
Throwable
├── Error
└── Exception
```

141. What is an Error?

Answer: An Error generally represents serious JVM or system-level problems that applications typically do not handle.

142. What is an Exception?

Answer: An exception represents conditions that application code may handle.

try-catch

143. What is a try block?

Answer: A try block contains code that may cause an exception.

144. What is a catch block?

Answer: A catch block handles a matching exception thrown from the associated try block.

145. Can we have multiple catch blocks?

Answer: Yes.

```
try {  
    // risky code  
}  
catch (ArithmeticException e) {  
}  
catch (Exception e) {  
}
```

The more specific exception should come before a more general one.

146. Can we have try without catch?

Answer: Yes, if it is followed by a finally block.

```
try {  
}  
finally {  
}
```

throw

147. What is throw?

Answer: throw is used to explicitly throw an exception object.

```
throw new Exception("Error");
```

148. Can we throw a custom exception?

Answer: Yes.

throws

149. What is throws?

Answer: throws declares exceptions that a method may propagate to its caller.

```
void read() throws IOException {  
}
```

150. Difference between throw and throws?

Answer:

throw → actually throws an exception.

throws → declares possible exceptions in a method signature.

finally

151. What is finally?

Answer: finally is a block normally used for cleanup code and is generally executed whether an exception occurs or not.

152. Is finally always executed?

Answer: Normally yes, but not in every conceivable situation— for example, abrupt JVM termination can prevent it from executing.

153. What is the common use of finally?

Answer: Resource cleanup, such as closing resources when not using try-with-resources.

Built-in Exceptions

154. What is ArithmeticException?

Answer: It occurs for invalid arithmetic operations, such as integer division by zero.

155. What is NullPointerException?

Answer: It occurs when an operation requiring a non-null reference is performed on a null reference.

156. What is ArrayIndexOutOfBoundsException?

Answer: It occurs when an invalid index is used to access an array.

157. What is `NumberFormatException`?

Answer: It occurs when a string cannot be converted to the requested numeric type.

Example:

```
Integer.parseInt("abc");
```

158. What is `ClassCastException`?

Answer: It occurs when an object is incorrectly cast to an incompatible reference type.

USER-CREATED EXCEPTIONS

159. What is a user-defined exception?

Answer: A user-defined or custom exception is an exception class created by the programmer for application-specific error conditions.

160. How do you create a custom exception?

Answer: Create a class extending `Exception` or another suitable exception class.

```
class AgeException extends Exception {  
    AgeException(String msg) {  
        super(msg);  
    }  
}
```

161. Why do we create custom exceptions?

Answer: To represent application-specific error conditions clearly and make error handling more meaningful.
